Data Visualisation Project - Python

Abraham Punkkaduthyil

21267755

Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.patches as patches
import matplotlib.pyplot as plt
import matplotlib.colors as cc
from matplotlib.cluster import MultipleLocator
```

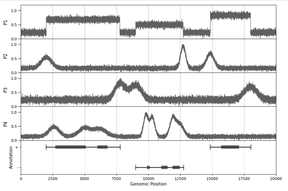
1. Visualization Exercise 1

Input

```
#load required data

df_annotation = pd.read_csv("T1_project_data_annotation.csv",
df_signalpep=pd.read_csv("T1_project_data_signals.csv",
df_line_plot=pd.read_csv("T1_project_data_timeplot.csv"
```

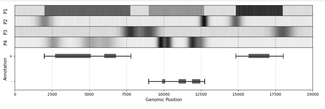
1.1.Version A

```
from matplotlib.patches import Rectangle

# Extract signals by creating a dictionary of signals (omits no seq and data an values
signals = {'P1':(df_signalp['P1:S1'] for i in range(1, 5)
# plot size
fig, axes = plt.subplots(5, 1, figsize=(12, 8), sharex=True, gridspec_kw={'height_ratios': [1] * 5})

# iterating the signals
for i, (label, signal) in enumerate(signals.items()):
    ax = axes[i]
    ax.plot(signal, color='#000000', linewidth=1)
    ax.set_ylim(-0.5, 1.5)
    ax.set_yticks([0.0, 0.5, 1.0])
    ax.set_yticklabels(['0.0', '0.5', '1.0'])
    ax.set_ylabel(label, rotation=0, labelpad=20, fontsize=11, ha='center', fontweight=0)
    ax.tick_params(axis='y', length=0,labelsize=8)

# Add vertical dotted lines
for line_x in [2500, 5000, 7500, 10000, 12500, 15000, 17500]:
    ax.axvline(line_x, color='#000000', linewidth=0.5, linestyle='dotted', alpha=0.5)

# Create free annotation plot as function to reuse in canvas plots
def plot_annotations(canvas, df_annotation, genome, alim):
    ax = canvas
    ax.set_xlim(0, alim)
    ax.set_ylim(-0.5, 0.5)
    ax.set_yticks([0.0])
    ax.set_ylabel("Annotation", fontsize=11
    ax.set_yticklabels(['Domain'], fontweight=0)
    ax.grid(axis='x', linewidth=0.5)

for gene in genome:
    gene_data = df_annotation[df_annotation['name'] == gene]
    if gene_data.empty:#if slice is the next gene if no data found
        continue
    y = 0.0 if gene_data.iloc[0]['strand'] == '+' else 0.2

    # Draw arrowhead
    start, stop = gene_data['start'].min(), gene_data['stop'].max()
    ax.plot([start, stop], [y, y], color='#000000', linewidth=1.5)
    ax.hlines(y, start - 50, stop + 50, linewidth=0.5, color='#333333')
    ax.hlines(y, stop, stop + 50, linewidth=0.5, color='#333333')

    # For genes
    for _, row in gene_data[gene_data['type'] == 'exon'].iterrows():
        ax.add_patch(Rectangle((row['start'], y - 0.05), row['stop'] - row['start'], 0.1, facecolor='#000000'))

# Add annotation plot
plot_gene_annotation(axes[4], df_annotation, ['gene1', 'gene2', 'gene3'], len(df_signals))

# Adjust spacing between plots
plt.subplots_adjust(hspace=0)
plt.show()
```



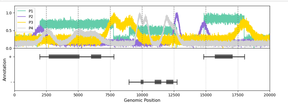
1.2. Version B

```
# signal dictionary which removed no seq plot
signals = {'P1':(df_signalp['P1:S1'] for i in range(1, 5)

# Create subplots
fig, axes = plt.subplots(5, 1, figsize=(14, 4), sharex=True, height_ratios=[1, 1, 1, 1, 4])

# Plot each signal on a backtrap
for i, (label, signal) in enumerate(signals.items()):
    ax = axes[i]
    ax.imshow(signal[:, np.newaxis], aspect='auto', vmin=0, vmax=1)  # Heatmap
    ax.set_ylabel(label, rotation=0, labelpad=20, fontsize=11, ha='center', rotation=90)  # Label
    ax.set_yticks([])  # Remove y-ticks
    ax.tick_params(axis='y', length=0, width=0, labelsize=0)  # Customize ticks

# Add vertical dotted lines
for line_x in [2500, 5000, 7500, 10000, 12500, 15000, 17500]:
    ax.axvline(line_x, color='#000000', linewidth=0.5, linestyle='dotted', alpha=0.5)

# Plot gene annotations on the last axes
plot_gene_annotation(axes[4], df_annotation, ['gene1', 'gene2', 'gene3'], len(df_signals))

# Adjust spacing between plots and display
plt.subplots_adjust(hspace=0)
plt.show()
```



1.3. Version C

```
fig = plt.figure(figsize=(12, 6))
gs = gridspec.GridSpec(5, 1, height_ratios=[1, 1])

ax0 = fig.add_subplot(gs[0])
ax1 = fig.add_subplot(gs[1], sharexax0)

# Plotting the same dictionary signals
signals = {'P1':(df_signalp['P1:S1'] for i in range(1, 5)
colors = {"hydrophobicity": "#e41a1c", "polarity": "#377eb8", "flipflops":}

# Plot each signal on the first axes (ax0)
for i, (label, signal) in enumerate(signals.items(), 1):
    ax0.plot(signal, label=f"Group {i}", color=colors[i])

# Configure ax0
ax0.set_ylim(0, 1.2)
ax0.set_yticks([0.0, 0.5, 1.0])
ax0.grid(axis='x', linestyle='dotted', linewidth=0.5)
ax0.legend(loc='upper left')
ax0.tick_parameters(axis='x', which='both', bottom=False, top=False, labelbottom=False)

# Add vertical lines
for line_x in [2500, 5000, 7500, 10000, 12500, 15000, 17500]:
    ax0.axvline(line_x, color='#000000', linewidth=0.5, linestyle='dotted', alpha=0.5)

# Calling the function and plot gene annotations on the second axes (ax1)
plot_gene_annotation(ax1, df_annotation, ['gene1', 'gene2', 'gene3'], len(df_signals))

# Adjust subplot spacing
plt.subplots_adjust(hspace=0)
plt.show()
```



1.4. Discussion

Version A:--Each protein has its own plot, making it easy to see their individual signal patterns without overlap. The annotation track is also clear and well-aligned. However, this version takes up a lot of space because each protein has a separate plot. Comparing P1-P4 is more challenging since they're in different plots, and using the same color for all makes it difficult to spot differences.

Version B:--All data is shown in one plot, saving space and making it easier to compare proteins and spot patterns. However, if the colors aren't chosen carefully, it can be hard to distinguish between the proteins.

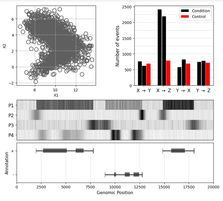
Version C: As mentioned earlier, all proteins are plotted together, saving space and making comparisons easier. However, overlapping lines can make it hard to see individual trends, especially in dense regions.

In my opinion, **Version B** would be a better choice because it balances clarity and comparisons. It's easy to understand and saves space while still showing patterns and comparisons well. Just make sure to use clear colors.

```
#Loading data
scatter_plot = pd.read_csv("T1_project_data_scatter.csv"
box_plot = pd.read_csv("T1_project_data_boxplot.csv"
```

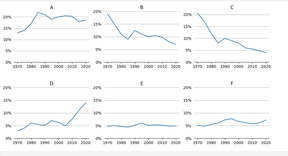
2. Visualization Exercise 2

```
from matplotlib.cluster import MultipleLocator
#Scatterplot data frame
df_scatter_plot.fillna(0)
# Create the figure and grid layout
fig, axes = plt.subplots(1, 2, sharexsharey=F)
fig.set_size_inches(8, 5)
gs = fig.add_gridspec(4, 4)  # 4x4 grid for subplots

# Define subplots
f_ax1 = fig.add_subplot(gs[0:2, 0:2])
f_ax2 = fig.add_subplot(gs[0:2, 2:4])
f_ax3 = fig.add_subplot(gs[2:4, 0:2])
f_ax4 = fig.add_subplot(gs[2:4, 2:4])

# Adjust layout spacing
fig.get_layout_engine().set_rect(rect=(0.025, 0.12, 0.975, 0.92))

# Scatter plot on f_ax1
f_ax1.scatter(df, s=2, marker='o', c=0.25, edgecolor='#000000', facecolor='none', linewidth=0.5)
f_ax1.set_xlim(-2, 12)
f_ax1.set_xlabel('X1')
f_ax1.set_ylim(-2, 6)
f_ax1.set_ylabel('X2', rotation=0)
f_ax1.grid(True, linestyle='dotted', linewidth=0.5)

# Box plot on f_ax2
labels = ['X1', 'X2', 'X3', 'X4']
boxplot_data = df[['X1', 'X2', 'X3', 'X4']]
boxplot_1 = f_ax2.boxplot(boxplot_data, widths=0.5, vert=True, labels=labels)

# Box plot on f_ax3
labels = ['X1', 'X2', 'X3', 'X4']
boxplot_data = df[['X1', 'X2', 'X3', 'X4']]
boxplot_2 = f_ax3.boxplot(boxplot_data, widths=0.5, vert=True, labels=labels)
contrast = f_ax3.get_xticklabels()

box_width = 0.2
box_gap = 0.1
index = np.arange(len(labels))

color_condition = 'black'
color_control = 'red'

barx1 = f_ax4.bar(index, condition_1, bar_width, color=color_condition, label='Condition', zorder=3)
barx2 = f_ax4.bar(index + box_width, control_1, bar_width, color=color_control, label='Control', zorder=3)
barx3 = f_ax4.bar(index + 2 * box_width + box_gap, control, bar_width, color=color_control, label='Control', zorder=3)

f_ax4.set_ylabel('Number of events', fontsize=11)
f_ax4.yaxis.set_minor_locator(MultipleLocator(250))
f_ax4.grid(which='both', axis='y', linestyle='dotted', linewidth=0.4, zorder=0)
f_ax4.set_xticks(index + box_width)
f_ax4.set_xticklabels(['X1 = Y', 'X2 = Z', 'Y = X', 'Y = Z'], fontsize=12)
f_ax4.tick_parameters(bottom=False, top=False, which='minor')
f_ax4.legend(loc='upper right')

# Heatmap on f_ax4
signals = {'P1':(df_signalp['P1:S1'] for i in range(1, 5)
# Convert signals to numpy array
heatmap_data = np.array(list(signals.values()))

heatmap = f_ax4.imshow(heatmap_data, cmap='Greys', aspect='auto', vmin=0, vmax=1,interpolation = 'none')

f_ax4.set_yticks(np.arange(len(label_list)))  # Set y-tick positions
f_ax4.set_yticklabels(label_list, fontsize=9)


# Remove y-axis ticks
f_ax4.set_xticks([])

# Save annotation as f_ax4
plot_gene_annotation(f_ax4, df_annotation, ['gene1', 'gene2', 'gene3'], len(df_signals))

# Display the figure
plt.show()
```

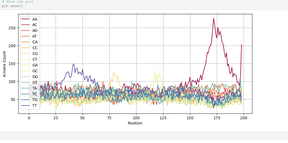


3. Visualization Exercise 3

```
import matplotlib.pyplot as plt

# Define the plot data function
def plot_data(ax, data, ax=):
    ax.set_xlim(1970, 2020)
    ax.set_ylim(ymin=0, ymax=20)
    ax.plot(data[data['year'] == 1970]['Year'] y ON ax.get_ylim())
    ax.grid(True, axis='y', linestyle='-')
    ax.tick_parameters(left=False,
    ax.spines['top'].set_visible=False,
    ax.spines['right'].set_visible=False,
    ax.spines['left'].set_visible=False,

# Create subplots
fig, axs = plt.subplots(2, 3)
fig.set_size_inches(8, 5)
fig.subplots_adjust(hspace=0.5, wspace=0.2)

# Subplot loop
years = df_line_plot["year"]
display_labels = [
label = ["A", "B", "C", "D", "E", "F"]

axs = axs.flatten()

# Loop through labels and plot the data
for idx, l in zenumerate(label):
    ax = axs[idx]
    ax.set_title(l)
    plot_data(Years, Data, ax=ax[l])

plt.show()
```



4. Visualization Exercise 4

```
#Creating a function to count the kmer in sequence
def count_kmer_occurrences(sequences, kmer):
    # Initialize a dictionary to store kmer counts
    kmer_counts = {}
    # Iterate through each sequence in the list
    for seq in sequences:
    # Calculate the length of the sequence
    seq_length = len(seq)
    # Iterate through the sequence
    for seq in sequences:
    for i in range(seq_length - k + 1):
        kmer = seq[i:i+k]
        if len(kmer) == k:
            kmer_counts[kmer] += 1
    return kmer_counts
```

```
import matplotlib.pyplot as plt
import numpy as np

df_line = pd.read_csv("T1_project_data_dna_sequences.csv", "r")
sequences = df_line.iloc[:, 0].tolist()  # Read line and remove newline characters
df_line.fillna(0)

# List of 3-mers that we want search
kmers = ['AAA', 'AC', 'AG', 'AT', 'CA', 'CC', 'CG', 'CT', 'GA', 'GC', 'GG', 'GT', 'TA', 'TC', 'TG', 'TT']

# Dictionary to store 3-mer counts
kmer_counts = {}

# Count occurrences of each 3-mer by calling the above function
for kmer in kmers:
    count = count_kmer_occurrences(sequences, kmer)
    kmer_counts[kmer] = count

# Sort the kmers by their counts
sorted_kmers = sorted(kmer_counts.items(), key=lambda x: x[1])
sorted_kmers = [kmer for kmer, count in sorted_kmers]

# Create a figure and axes
fig, ax = plt.subplots(figsize=(12, 4.7))

# Add "Spectral" colormap with 16 distinct colors
cmap = plt.get_cmap("Spectral", 16)

# Plot the results
for i, (kmer, count) in enumerate(kmer_counts.items()):
    # Choose a color from the colormap based on the index of kmer
    color = cmap(i / 15)  # Divide by 15 to get values between 0 and 1

    # Create a line plot with the selected color
    ax.plot([0, 1], [kmer, kmer], color=color)

# Set labels, title, and legend
ax.set_xlabel('Position')
ax.set_ylabel('3-mers (kmer)')
ax.legend(loc='upper left', bbox_to_anchor=(1.0, 1), ncol=2)  # 2 columns for legend

# Show the plot
plt.show()
```